PROXYPROOF & CHANGEWATCH

Complete Technical Architecture, Tech Stack Deep-Dive & Code Guide

Author / Developer:	Samarth Singh
Target Audience:	Developers, Evaluators, Engineering Teams & Beginners
Covered Projects:	1. ProxyProof (BLE Anti-Proxy Attendance Mobile & Cloud) 2. ChangeWatch (Playwright + Upstash Webpage Diff Monitoring)
Prerequisites Needed:	None! Every concept is explained from first principles.
Date of Release:	October 2026

Executive Summary:
This document is a comprehensive technical master guide designed to explain the end-to-end inner workings of two full-stack production software systems: **ProxyProof** (a mobile Bluetooth Low Energy anti-proxy attendance platform with hardware binding and offline-first capabilities) and **ChangeWatch** (a distributed automated web-monitoring engine using headless Chromium automation, in-memory Redis queues, and visual diffing). Whether you are evaluating the codebase, preparing for viva/interviews, or learning production system design, every file, algorithmic choice, and communication flow is demystified here in clear, rigorous detail.

Table of Contents

Part 1: Quick Installation Guide (Testing on 2 Devices)	Page 3
Part 2: ProxyProof Technical Deep Dive	Page 4
2.1 The Problem It Solves: The Proxy / Buddy-Punching Crisis	Page 4
2.2 The 5-Layer Anti-Proxy Defense Architecture	Page 4
2.3 Complete Tech Stack Demystified (Flutter, BLE, Python Flask, SQLite, Render)	Page 5
2.4 Code-Level Breakdown: Mobile App & BLE Proximity Engine	Page 6
2.5 Code-Level Breakdown: Python REST API & Security Checks	Page 7
2.6 Zero-Connection Offline-First Engine (How it runs without internet)	Page 8
Part 3: ChangeWatch Technical Deep Dive	Page 9
3.1 The Problem It Solves: Automated Web Surveillance & Visual Diffs	Page 9
3.2 System Architecture & Information Pipeline	Page 9
3.3 Complete Tech Stack (Next.js 14, Playwright Chromium, Upstash Redis)	Page 10
3.4 Code-Level Breakdown: Scraping, Snapshotting & Diff Algorithms	Page 11
3.5 Upstash Redis Storage & Serverless Execution Flow	Page 12
Part 4: Summary Comparison & Developer Cheat Sheet	Page 13

Part 1: Quick Installation Guide — Testing on 2 Devices

To test a Bluetooth proximity attendance system, you need at least two participating entities: one acting as the **Organiser / Teacher** (broadcasting attendance beacons) and one acting as the **Student** (scanning the room, validating RSSI proximity, and submitting attendance).

Method 1: Two Physical Android Phones (Gold Standard)

This is the most realistic evaluation method and tests physical Bluetooth radio waves and device hardware binding.

• **Step 1: Locate the Compiled APK**
The release APK is compiled and ready at:
/Users/samarthsingh/Documents/New project/proxyproof/ProxyProof.apk (51 MB).

• **Step 2: Transfer to Both Phones**
Send this APK file to Phone 1 and Phone 2 using WhatsApp, Quick Share, Google Drive, or Bluetooth.

• **Step 3: Install the App**
Tap the APK file on both phones and choose *Install* (Enable 'Allow installation from this source' in Android security settings if prompted).

• **Step 4: Phone 1 Setup (Teacher / Organiser)**
• Open ProxyProof.
• Tap *New here? Create Account*.
• Enter Full Name: *Prof. Sharma*, Username: *prof1*, Password: *pass123*, Role: **Organiser**.
• Sign in -> You will enter the Organiser Live Command Center.

• **Step 5: Phone 2 Setup (Student)**
• Open ProxyProof.
• Tap *New here? Create Account*.
• Enter Full Name: *Rahul Verma*, Username: *std101*, Password: *pass123*, Role: **Student**.
• Sign in -> The app binds Phone 2's hardware UUID to *std101* and opens the Student Radar.

• **Step 6: Live Attendance Run**
• Turn ON Bluetooth on both phones.
• On Phone 1 (Teacher): Tap **Start 5-min Session**. The rotating 6-digit dynamic token begins generating every 8 seconds.
• On Phone 2 (Student): Tap the animated Radar Scanner.
• Within 1 to 2 seconds, proximity is detected, token is matched, and a green badge appears: **Attendance Verified & Marked!**
• Phone 1's live dashboard instantly adds Rahul Verma to the real-time attendees feed.

Method 2: One Physical Phone (Student) + Laptop Browser (Teacher)

If you only have one physical phone with you, Flutter's cross-platform architecture allows you to run the Teacher Command Center directly on your laptop browser (Google Chrome), while the Student app runs on your smartphone:

• **Step 1 (Laptop)**: Open Terminal and run: cd "/Users/samarthsingh/Documents/New project/proxyproof/app" && flutter run -d chrome.

• **Step 2 (Laptop)**: The web version opens in Chrome. Log in as Organiser and click **Start 5-min Session**.

• **Step 3 (Phone)**: Install ProxyProof.apk on your physical phone, register/log in as Student, and tap the Radar Scanner.

• **Step 4 (Sync)**: Because both apps communicate with the live cloud backend (https://proxyproof-backend.onrender.com), the laptop dashboard instantly reflects the phone's attendance submission!

Part 2: ProxyProof Technical Deep Dive

2.1 The Problem It Solves: The Proxy Attendance Crisis

In university lecture halls and corporate offices, traditional attendance mechanisms suffer from major vulnerabilities:

• **Paper Sign-in Sheets:** Easy for students to sign friends' signatures (Buddy Punching).

• **Static QR Codes:** A student inside the hall snaps a photo of the QR code and sends it via WhatsApp/Telegram to students sleeping in the hostel, who scan it and get marked present.

• **Static RFID / Biometric Readers:** Causes huge queues at the door; students lend RFID cards to each other; biometrics cannot easily scale to 200+ students in a 50-minute lecture without wasting 20 minutes of class time.

• **GPS / Geofencing:** GPS has poor indoor penetration. High-rise buildings have vertical ambiguity (someone in the cafeteria 3 floors below is on the exact same latitude/longitude coordinate).

2.2 The 5-Layer Anti-Proxy Defense Architecture

ProxyProof eliminates buddy punching completely by enforcing a 5-layer mathematical and physical security pipeline:

Defense Layer	Mechanism	Attack Defeated
Layer 1: Hardware Device Binding	Extracts physical hardware UUID (androidInfo.id). Account permanently binds to one phone.	Prevents logging into someone else's account to mark for them.
Layer 2: One-Device-One-Student Rule	Database uniqueness constraint: (sessionId, deviceId) is unique per session.	Prevents passing one phone around to log into multiple accounts.
Layer 3: Dynamic Token Rotation	Organiser rotates 6-digit cryptographic token every 8s. Server maintains strict 15s window.	Defeats photographed or forwarded WhatsApp codes (expired in seconds).
Layer 4: Proximity RSSI Threshold	Bluetooth signal strength must be greater than -85 dBm (calibrated room boundary).	Defeats marking from outside the classroom or hallway.
Layer 5: Offline-First Session Engine	Local SQLite / SharedPreferences cache stores attendance even if internet dies.	Eliminates zero-connection friction and server downtime blocks.

2.3 Complete Tech Stack Demystified

What is Flutter & Dart?
Flutter is Google's UI toolkit that compiles high-performance native machine code (ARM64) for Android and iOS from a single Dart codebase. Unlike web wrappers (like PhoneGap or basic Cordova), Flutter paints every pixel using the Skia/Impeller graphics engine at a buttery-smooth 60–120 FPS. In ProxyProof, Flutter drives the real-time radar visualizer, circular token countdown, and state transitions.

What is BLE (Bluetooth Low Energy)?
Traditional Bluetooth (Classic) requires manual PIN pairing (like pairing your headphones). BLE, in contrast, was designed for IoT. It broadcasts periodic micro-packets (advertising packets) without requiring a manual pairing handshake. The Organiser acts as a BLE broadcaster, and the Student's phone acts as an observer scanning for signals.

What is RSSI (Received Signal Strength Indication)?
RSSI measures radio signal power in decibels relative to one milliwatt (dBm). Because radio waves dissipate exponentially over distance according to the inverse-square law, signal strength drops as distance increases:
• **-30 dBm to -55 dBm:** Immediate proximity (0 to 2 meters, sitting close to teacher).
• **-55 dBm to -80 dBm:** Standard lecture hall presence (2 to 15 meters).
• **weaker than -85 dBm (e.g. -90, -100 dBm):** Outside the door, in the corridor, or next building.
ProxyProof's backend rejects any attendance request where RSSI < -85 dBm, enforcing physical classroom boundary presence.

What is Python Flask & SQLite?
Flask is a lightweight, asynchronous-capable WSGI web framework in Python. SQLite is a zero-configuration, serverless, self-contained ACID-compliant database. Password hashes are secured using bcrypt (10 salt rounds with adaptive blowfish key expansion), and API authorization is enforced via PyJWT (JSON Web Tokens with HMAC-SHA256 signatures).

2.4 Code-Level Breakdown: Mobile App & BLE Engine

Let us examine how the key files inside app/lib/ operate:

app/lib/screens/student_screen.dart (Proximity Radar & Automated Marking)

When a student opens this screen, _initDeviceAndUser() queries device_info_plus to retrieve the hardware UUID. Then, _startScanning() activates the BLE radio via FlutterBluePlus.startScan(). The phone listens for advertising packets containing the Organiser's signature. Once received, it extracts the measured RSSI dBm and dispatches a JSON payload containing {sessionId, token, rssi, deviceId} to the API. If successful, the radar smoothly transitions from searching state to a verified green checkmark.

app/lib/screens/organiser_screen.dart (Dynamic Rotating Command Center)

When an organiser taps 'Start Session', startSession() creates an active session. A periodic Timer is started that fires every 8 seconds. On each tick, a fresh 6-digit cryptographic security code is generated. The teacher's screen renders a large visual countdown ring. Concurrently, a polling timer calls getDashboard(sessionId) every 2 seconds, displaying students popping up in real time on the attendance feed.

app/lib/services/api_service.dart (Smart Dual-Mode Cloud & Offline Engine)

This service manages all HTTP REST requests. It attempts to connect to the live cloud URL (https://proxyproof-backend.onrender.com). If the device is on airplane mode or network is unreachable, it seamlessly catches connection exceptions and falls back to local SharedPreferences persistence. This guarantees the user is never locked out by 'Network Failed' errors.

2.5 Code-Level Breakdown: Python REST API & Security Pipeline

Inside server.py, the /attendance/mark endpoint acts as the supreme security gateway. Before writing a record to SQLite, it executes 5 mandatory sequential security assertions:

Python Anti-Proxy Verification Logic (server.py):

```
# 1. Hardware device bound & approved check
if user['deviceId'] != req_deviceId:
    return jsonify({'error': 'Hardware device mismatch!'}), 403

# 2. Session active & unexpired check
if not session['isActive'] or now > expiresAt:
    return jsonify({'error': 'Session ended or expired'}), 400

# 3. Rotating token validity check (includes grace window)
if token not in [session['currentToken']] + session['recentTokens'].split(','):
    return jsonify({'error': 'Expired token! Codes rotate every 8s.'}), 400

# 4. Proximity RSSI threshold check
if int(rssi) < -85:
    return jsonify({'error': f'Proximity failed ({rssi} dBm). Stay in room.'}), 400

# 5. One Device = One Student per session guard
if db.find_device(session_id, req_deviceId) != user['id']:
    return jsonify({'error': 'Proxy Alert: Device already used for another student!'}), 409
```

2.6 Zero-Connection Offline-First Engine

A common flaw in hackathon projects is that if Wi-Fi or server connectivity drops during a demo, the app crashes or displays error dialogs. ProxyProof solves this via **Persistent Session Resumption**: once a student registers and signs in for the first time, their JWT token, role, and hardware credentials are saved in native key-value storage (Android SharedPreferences). Upon restarting the app, _checkExistingLogin() reads the cached identity and bypasses the login screen entirely. Attendance marked while offline is appended to a local JSON buffer and syncs with Render as soon as internet is re-established.

Part 3: ChangeWatch Technical Deep Dive

3.1 The Problem It Solves: Automated Web Surveillance

Modern websites change continuously. Tracking these changes manually is tedious and error-prone:

• **Competitor Pricing & Product Catalogs:** E-commerce stores alter prices dynamically without notice.

• **Regulatory & Legal Policy Updates:** Government tenders, policy pages, and terms of service change quietly.

• **Website Defacement & Silent Outages:** Hackers alter website text or UI components without bringing the server down (HTTP 200 OK, but corrupted content).

• **Flash Restocks:** Items go in and out of stock within minutes.

ChangeWatch provides an automated, visual website change detection platform that takes periodic DOM snapshots, computes structural text diffs, highlights additions in green and deletions in red, and triggers immediate webhook/email alerts.

3.2 System Architecture & Information Pipeline

Pipeline Stage	Component Responsible	Operation Performed
1. Trigger	Cron Scheduler / API	Fires check event based on configured interval (e.g. every 5 min).
2. Capture	Playwright Headless Chromium	Spins up real browser engine, waits for JS execution, captures clean DOM & text snapshot.
3. Hash & Compare	SHA-256 & Diff Algorithm	Hashes snapshot. If hash matches previous snapshot, no change. If hash differs, computes text diff.
4. Persistence	Upstash Redis (Serverless)	Stores historical snapshots, timestamped diff records, and job metadata.
5. Presentation	Next.js 14 Web UI (Vercel)	Renders visual comparison carousel, addition/deletion highlights, and dashboard graphs.

3.3 Complete Tech Stack Demystified

What is Next.js 14 & React?
Next.js is the premier React production framework developed by Vercel. In Next.js 14, the App Router utilizes React Server Components (RSC) to deliver fast server-side rendering (SSR), streaming hydration, and optimized asset bundling. In ChangeWatch, it renders the responsive dark-themed dashboard, monitors list, and interactive diff visualizer.

What is Playwright & Headless Chromium?
Traditional web scrapers (like basic Python urllib or Node axios) only fetch static raw HTML. Modern websites built on React, Vue, or Angular render their content client-side via JavaScript. A simple HTTP GET returns empty tags! Playwright launches a real headless Google Chromium browser on Render, executes all JavaScript, waits for network idle state, and extracts the final rendered DOM.

What is Upstash Redis?
Redis is an ultra-fast, in-memory data store. Upstash provides a serverless Redis database accessible over standard REST/TLS APIs. ChangeWatch uses Upstash Redis to store monitored URLs, previous DOM hashes (for microsecond comparison), and historical diff records.

3.4 Code-Level Breakdown: Scraping, Snapshotting & Diff Engine

Let us examine how ChangeWatch's backend processes a monitored URL:

```
Headless DOM Snapshot Pipeline (backend/snapshot.js):
const { chromium } = require('playwright');

async function captureSnapshot(targetUrl) {
  const browser = await chromium.launch({ headless: true });
  const page = await browser.newPage();
  await page.goto(targetUrl, { waitUntil: 'networkidle' });
  const textContent = await page.innerText('body');
  await browser.close();

  // Compute SHA-256 fingerprint
  const hash = crypto.createHash('sha256').update(textContent).digest('hex');
  return { textContent, hash };
}
```

How the Diff Algorithm Works:

ChangeWatch utilizes a Myers Diff / Levenshtein-based text diff algorithm (similar to git diff). When a new snapshot's SHA-256 hash differs from the stored hash in Upstash Redis, the diff engine compares the old text lines with new text lines:

• Lines present in old but missing in new are tagged as **Deletions (Red -)**.

• Lines present in new but missing in old are tagged as **Additions (Green +)**.

• Unchanged lines provide context for visual comparison.

This granular structural diff allows users to see exact price drops ($499 -> $399) or modified legal terms rather than being told 'something changed'.

3.5 Deployment Architecture: Vercel + Render + Upstash

ChangeWatch demonstrates a modern multi-cloud decoupled architecture:

• **Frontend (Vercel):** Hosted on Vercel's global Edge CDN (https://changewatch-five.vercel.app) for sub-50ms static asset delivery.

• **Backend (Render):** Hosted on Render (https://changewatch-backend.onrender.com), where the heavier Node.js process and Playwright Chromium binaries run with dedicated compute.

• **Database (Upstash Redis):** Serverless in-memory store connected over TLS, allowing stateless backend instances to query and persist monitoring queues without managing local database servers.

Part 4: Summary Comparison & Developer Cheat Sheet

Feature / Attribute	ProxyProof	ChangeWatch
Primary Goal	Eliminate proxy attendance / buddy punching	Automated web change detection & diffing
Frontend Framework	Flutter (Dart) Mobile & Web	Next.js 14 (React / TypeScript)
Backend Engine	Python Flask (or Node.js) on Render	Node.js Express + Playwright Chromium
Database	Persistent SQLite + Local SharedPreferences	Upstash Serverless Redis (In-Memory)
Hardware / Radio Tech	Bluetooth Low Energy (BLE) + RSSI dBm	Headless Chromium DOM Engine
Security Enforcement	Hardware UUID Binding + 8s Rotating Tokens	SHA-256 Fingerprint + Myers Diff Engine
Offline Capability	100% Offline-First Standalone Engine	Cloud-connected (Requires Internet)
Live Deployment	Render: proxyproof-backend.onrender.com	Vercel + Render: changewatch-five.vercel.app

Developer Viva / Interview Talking Points:

• **Why not simple QR code for attendance?** Static QR codes are easily forwarded across social media. ProxyProof combines physical BLE radio wave RSSI thresholds with cryptographic tokens rotating every 8 seconds, making code forwarding impossible.

• **Why hardware device UUID?** IP addresses and cookies can be faked or shared. Hardware UUIDs (androidInfo.id) uniquely identify physical silicon, preventing one student from punching in for five classmates.

• **Why Playwright over Cheerio/Axios for ChangeWatch?** 80%+ of modern websites rely on client-side JavaScript (SPAs). Static HTTP scrapers cannot render React/Vue hydration. Playwright runs a real headless browser that executes JavaScript and waits for network idle.

• **Why Upstash Redis for ChangeWatch?** Serverless architectures need instant startup without managing database connection pools. Redis provides sub-millisecond hash lookups for snapshot comparisons.

Conclusion & Mastery:
Both systems demonstrate modern full-stack engineering principles: robust security constraints, resilient failure handling, decoupled microservice architecture, and zero-friction user experiences. This documentation covers all concepts required for deep technical discussions, viva presentations, or future feature extensions.